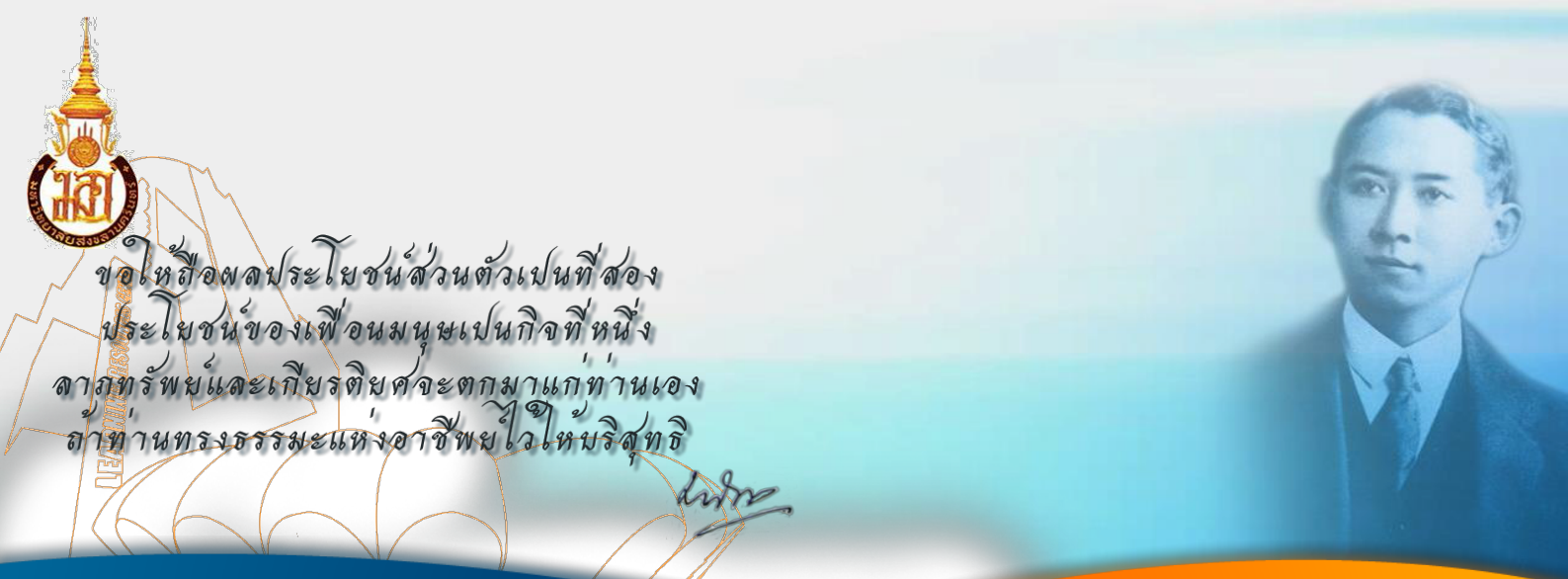


344-111

Dynamic Programming

Dynamic Programming

- **Idea:** Characterizing subproblems using a small set of integer indices – to allow an optimal solution to a subproblem to be defined by the combination of solutions to even smaller subproblems
- **Solution:** Solving each of smaller subproblems only once and recording the results in a table from which can obtain a solution to the original problem – **Using a table instead of recursion**

ตัวอย่าง Dynamic Programming

- 0-1 Knapsack problem
- All-pair short path
- Optimal binary search tree

In Class

- Longest increasing subsequence
- Partition Problem

Self-learning

0-1 Knapsack problem

Problem Formulation:

- 0-1 : reject or accept
- given **n** items of weights w_1 , w_2 ,…, w_n and values v_1, v_2 ,…, v_n and a knapsack of capacity W
- find the best solution that gives maximum weight and value

เลือกสิ่งของลงในถุงเป้ ให้ได้มูลค่ามากที่สุด
โดยที่ผลรวมของน้ำหนักไม่เกิน W

0-1 Knapsack problem

Example: let W = 5 and

weight	value	i	Capacity j (w=5)				
			1	2	3	4	5
w1=2	v1=12	1	0	12	12	12	12
w2=1	v2=10	2	10	12	22	22	22
w3=3	V3=20	3	No 10	12	22	30	32
w4=2	v4=15	4	No 10	15	25	30	37

แก้ไขมูลค่ารวม
ของแต่ละ sub problem

ดูตั้นกำเนิดค่า

i = 1
1 + 2
10 + 12
i = 2, 3
i = 4
i = 3, 3
37 = 15 + 22
∴ เลือก {1, 2, 4}

Maximum value

ใส่ใน capacity 1 ไม่ได้

เอาค่าข้าง

ใส่ได้

ใส่ได้ 3-2=1
ไปดู capacity 1
subproblem
ก่อนหน้า

ใส่ได้ 4-2=2
ไปดู15+12=27
น้อยกว่าเดิมไม่เอา

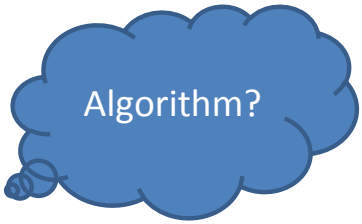
5-2=3
15+22 = 37 เอา

Selected items = 1,2,4

0-1 Knapsack problem

Solution:

- use table to fill in by applying formulas



$$B[i,j] = \begin{cases} B[i-1,j] \text{ , if } w_i > j & \text{ใส่ไม่ได้} \\ \max\{B[i-1,j] \text{ , } B[i-1,j-w_i]+v_i\} \text{, else} & \text{ใส่ได้ } (W_i <= j) \end{cases}$$

Algorithm: 0-1 Knapsack problem

```
0-1Knapsack(w1….wn, v1…vn, W, n)
{
    for each item i = 1…n      cosider first item to n
        for each j = 1…W
            if (wi > j)
                    b[i,j] = b[i-1,j]
            else
                    b[i,j] = max{b[i-1,j] , b[i-1,j-wi]+vi
            end if
        end for
    end for
    return b[n,w]
}
```

Time Complexity=??

subproblem

Maximum value

All-Pair Short Path

Problem Formulation:

- **Given:** G=(V,E) เป็นกราฟแบบไม่มีทิศทาง n โหนด

  $E = \{e_{i,j}\}$; i=1,…,n; j=1,…,n แทนระยะทางจากโหนด i ไป j

- **Goal:** หาเส้นทางที่สั้นที่สุดจากโหนด i ไปโหนด j ใดๆ

Solution: ให้ $d^k(i,j)$ คือ ระยะทาง short test path จาก i ไป j ผ่าน k ใดๆ

จะได้ $\mathbf{d^k(i,j) = \min(d^{k-1}(i,j), d^{k-1}(i,k)+d^{k-1}(k,j)); k = 1, \ldots, n}$

All-Pair Shortest Path

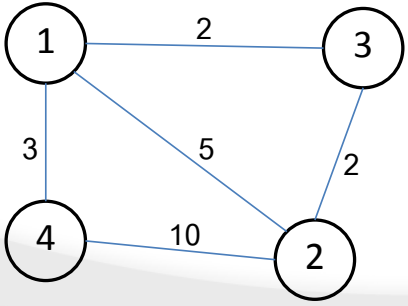
Example: กำหนดให้ G แทนกราฟดังนี้

1->3->2->4 = 14
1->2->4 = 15
1->4 = 3

เส้นทางที่สั้นที่สุดจากโหนด 1 ไปโหนด 4 คือ 1 -> 4

1->2 = 5

เส้นทางที่สั้นที่สุดจากโหนด 1 ไปโหนด 2 คือ 1->3->2 = 2 + 2 = 4



array 2D

e(j,j)	1	2	3	4
1	0	5	2	3
2	5	0	2	10
3	2	2	0	∞
4	3	10	∞	0

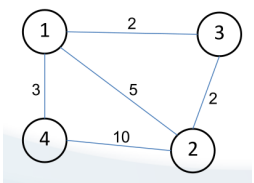
All-Pair Shortest Path

array 2D

e(j,j)	1	2	3	4
1	0	5	2	3
2	5	0	2	10
3	2	2	0	∞
4	3	10	∞	0

เส้นทางใหม่

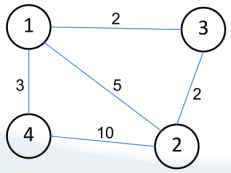
ระยะทางที่สั้นที่สุดจาก i ไป j ผ่าน k=1 (**$D^{(1)}[i,j] = \min(D^{(0)}[i,j] , D^{(0)}[i,1] + D^{(0)}[1,j])$)



$D^{(1)}(i,j)$	1	j= 2	j= 3	j= 4
1	0	Min(5,0+5)=5 1->1->2	Min(2,0+2)=2 1->1->3	Min(3,0+3)=3 1->1->4
i= 2	5	0	Min(2,5+5)=2	Min(10,5+3)=8
i= 3	2	2	0	Min(∞, **2 + 3**)=5
i= 4	3	8	**5**	0

2->1->4

All-Pair Shortest Path



array 2D

e(j,j)	1	2	3	4
1	0	5	2	3
2	5	0	2	10
3	2	2	0	∞
4	3	10	∞	0

ค่าใหม่เมื่อผ่าน node 2

ระยะทางที่สั้นที่สุดจาก i ไป j ผ่าน k=2 **$D^{(2)}[i,j] = \min(D^{(1)}[i,j], D^{(1)}[i,2] + D^{(1)}[2,j])$

1 -> 2 -> 3 = 7

1 -> 2 -> 4 = 13

2 -> 2 -> 4 = 10

3 -> 2 -> 4 = 12

d(i,j)	1	2	3	4
1	0	Min(5,5+0)=5	Min(2,5+2)=2	Min(3,5+8)=3
2	5	0	Min(2,0+2)=2	Min(8,0+8)=8
3	2	2	0	Min(**5**,2+8)=5
4	3	8	**5**	0

ค่าเดิมยังคงน้อยกว่า

All-Pair Shortest Path

ระยะทางที่สั้นที่สุดจาก i ไป j ผ่าน k=3 **$D^{(3)}[i,j] = \min(D^{(2)}[i,j] , D^{(2)}[i,3] + D^{(2)}[3,j])$

d(i,j)	1	2	3	4
1	0	Min(5,2+2)=4	Min(2,2+0)=2	Min(3,2+5)=3
2	4	0	Min(2,2+0)=2	Min(8,2+5)=7
3	2	2	0	Min(**5**,0+5)=5
4	3	7	**5**	0

All-Pair Shortest Path

ระยะทางที่สั้นที่สุดจาก i ไป j ผ่าน k=4 $**D^{(4)}[i,j] = \min(D^{(3)}[i,j] , D^{(3)}[i,4] + D^{(3)}[4,j])$

1 -> 4 -> j

x+y < ค่าเดิม

d(i,j)	1	2	3	4
1	0	Min(4,3+3)=4	Min(2,3+5)=2	Min(3,3+0)=3
2	4	0	Min(2,7+5)=2	Min(7,7+0)=7
3	2	2	0	Min(5,5+0)=5
4	3	7	**5**	0

Algorithm: All-Pair Shortest Path

```
AllPairShortestPath_DP(w[1..n,1..n])
{
        d[1..n,1..n] = w[1..n,1..n]
        for (k= 1..n){
            for (i=1..n){
                for (j=1..n)
                    dk[i,j] = min(d[i,j], d[i,k] + d[k,j]);
            }
            d[1..n,1..n] = dk[1..n,1..n]
        }
        return d[1..n,1..n]
}
```

Complexity=??

Optimal Binary Search Tree

Problem Formulation:

- **Given:** $w_1, \ldots, w_n$ is a set of words with its probabilities to be appeared $p_1, \ldots, p_n$ respectively, where $\sum_{i=1}^{n} p_i = 1$
- For example

word	probabilities
a	0.22
am	0.18
and	0.20
egg	0.05
if	0.25
the	0.02
two	0.08

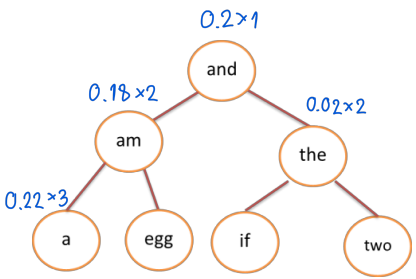
→ appear or stored data depent on define

Optimal Binary Search Tree

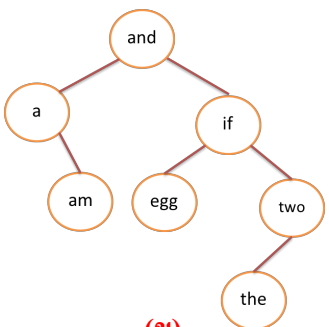
Goal: หาวิธีการจัดเก็บข้อมูลเหล่านี้ใน Binary Tree จึงจะมีต้นทุนในการค้นหาน้อยที่สุด (Cost)

ตัวอย่าง:

word	probabilities
a	0.22
am	0.18
and	0.20
egg	0.05
if	0.25
the	0.02
two	0.08



(ก)



(ข)

probabilities ↓ compare round

- ถ้าจัดเก็บในต้นไม้แบบ (ก) จะมีต้นทุนการค้นหาเป็น 0.2x1 + (0.18+0.02)x2 + (0.22+0.2+0.25+0.08)x3 = 2.4
- ถ้าจัดเก็บในต้นไม้แบบ (ข) จะมีต้นทุนการค้นหาเป็น 0.2x1 + (0.22+0.25)x2 + (0.18+0.05+0.08)x3+0.02x4=2.15

Optimal Binary Search Tree

- กำหนดให้ C(i,j) คือต้นทุนการค้นหาที่เหมาะสมที่สุด ซึ่งเก็บข้อมูล $x_i, x_{i+1},\ldots,x_j$ และมี x_m เป็นราก ($i <= m <= j$)
- ต้นทุนการค้นหาทั้งต้นจึงเท่ากับ

$$p_m + \left(C(i,m-1) + \sum_{i \le k \le m-1} p_k \right) + \left(C(m+1,j) + \sum_{m+1 \le k \le j} p_k \right) = C(i,m-1) + C(m+1,j) + \sum_{i \le k \le j} p_k$$

- Solution:

$$C(i,j) = \min_{i \le m \le j} \left(C(i,m-1) + C(m+1,j) \right) + \sum_{i \le k \le j} p_k$$

Optimal Binary Search Tree

Example: หา Optimal Binary Search Tree สำหรับข้อมูลดังตาราง

input

i	1	2	3	4	5	6	7
w	a	am	and	egg	if	the	two
p	0.22	0.18	0.20	0.05	0.25	0.02	0.08

$$C(i,j) = \min_{i \le m \le j} \left(C(i,m-1) + C(m+1,j) \right) + \sum_{i \le k \le j} p_k$$

p(i,j)	1	2	3	4	5	6	7
1	0.22	0.40	0.60	0.65	0.90	0.92	1.00
2		0.18	0.38	0.43	0.68	0.70	0.78
3			0.20	0.25	0.50	0.52	0.60
4				0.05	0.30	0.32	0.40
5					0.25	0.27	0.35
6						0.02	0.10
7							0.08

C(i,j)	1	2	3	4	5	6	7
1	0.22	0.58	1.02	1.17	1.83	1.89	2.15
2		0.18	0.56	0.66	1.21	1.27	1.53
3			0.20	0.30	0.80	0.84	1.02
4				0.05	0.35	0.39	0.57
5					0.25	0.29	0.47
6						0.02	0.12
7							0.08

3 Node

C(1,3) i=1, j=3 = 0.42 + 0.6 = 1.02

① C(2,1) 0.36 ② C(1,1) 0.22 C(3,3) 0.2 = 0.42 min → Root ③ C(1,2) 0.58

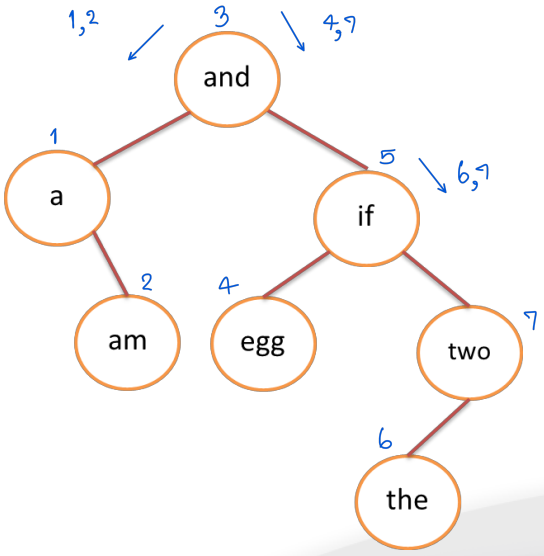
4 Node

C(1,4) = 0.52 + 0.65 = 1.17 ① (2,4) 0.66 ② C(1,1) C(3,4) 0.22 + 0.3 = 0.52 ③ C(1,2) C(4,4) 0.38 + 0.05 = 0.63 ④ C(1,3) 1.02

Create OBST

i	1	2	3	4	5	6	7
w	a	am	and	egg	if	the	two
p	0.22	0.18	0.20	0.05	0.25	0.02	0.08

r(i,j)	1	2	3	4	5	6	7
1	root 1	1	2	2	3	3	(3)
2		2	3	3	3	3	3
3			3	3	5	5	5
4				4	5	5	(5)
5					5	5	5
6						6	7
7							7



Algorithm: Create OBST

```
OptimalBST(x[1..n], p[1..n])
{
      r = optimalBST_DP(p[1..n])
      t = createBST(r, x)
}

createBST(r[1..n,1..n], x[i..j])
{
    if(i>j) return null
    m= r[i,j]
    left = createBST(r[1..n,1..n],x[i,m-1])
    right = creatBST(r[1..n,1..n],x[m+1,j])
    t = new node(x[m], left, right)
    return t
}
```

Algorithm: Create OBST

```
optimalBST_DP(p[1..n])
{
    for (i=1..n)  pp[i,i] = p[i]
    for (i=1..n-1)
        for (j=i+1..n)
            pp[i,j]  = pp[i,j-1] + p[j]
    for (i=1..n)  c[i,i-1] = 0
    for (k=1..n){
        for(i=1...n-k+1){
            j = i+k-1
            c[i,j] = inf
            for (m=i..j){
                newCost = c[i,m-1] + c[m+1,j] + pp[i,j]
                if (newCost < c[i,j]){
                    c[i,j] = newCost
                    r[i,j] = m
                }
            }
        }
    }
    return r
}
```

อ้างอิง

- การออกแบบและวิเคราะห์อัลกอริทึม, สมชาย ประสิทธิจตรกูล
- Introduction to the design and analysis of algorithm, Anany Levitin